



The Complete Guide to Agentic AI Design Patterns

Understanding How AI Agents Think and Act

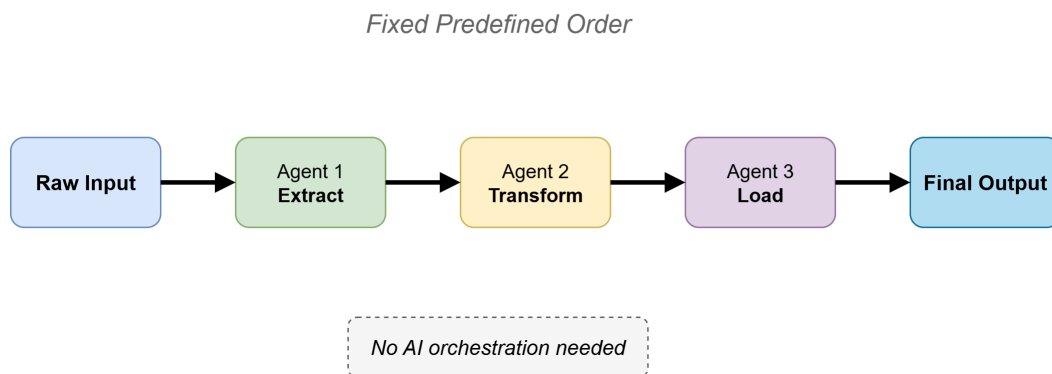
Building an AI agent isn't just about connecting a language model to some tools. The real challenge is choosing the right **architecture pattern**—the way your agent reasons, acts, and iterates to solve problems. Pick the wrong one, and you'll either over-engineer a simple task or leave a complex problem broken.

This guide breaks down each pattern with clarity: what it does, when to use it, and where it fails.

1. Sequential Workflow Pattern

For tasks with a known, predictable path

Sequential Pattern Example



What It Does

A sequential workflow is the simplest pattern: your agent follows explicit steps in order, moving from one stage to the next. Think of it as a production line—each step knows exactly what comes before and after.

Example: Invoice processing → Extract fields → Validate data → Store in database → Send confirmation

When to Use It

- ✓ The process is fully defined before execution begins
- ✓ Edge cases are rare or handled by deterministic code
- ✓ You need fast, predictable results
- ✓ Cost matters (minimal model calls)

The Trade-Off

You don't get flexibility. If the workflow encounters something unexpected, it breaks. The workflow is rigid by design—which is exactly what you want when the task is simple and stable.

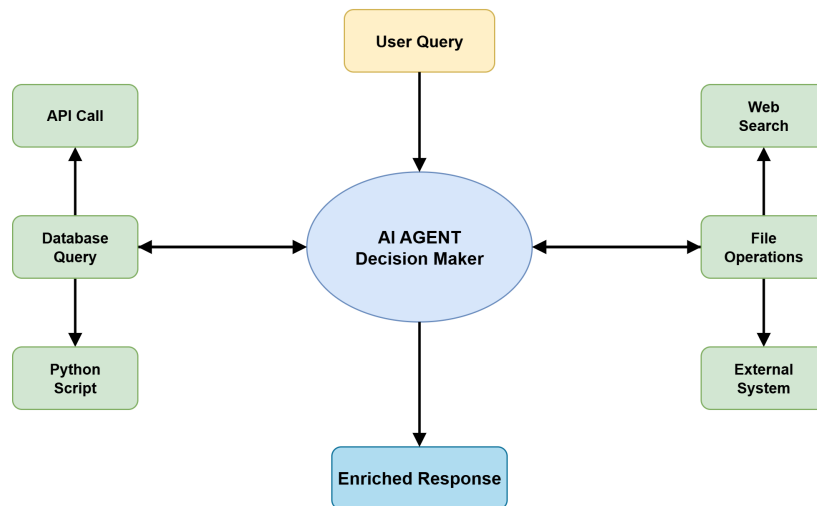
Best Practice

Let code handle execution; use the model only for reasoning and generation. Separate deterministic steps from decision points. This keeps systems fast and cheap.

2. Tool Use Pattern

For agents that need to access external information and systems

Tool Use Pattern



What It Does

Tool use is the foundation that everything else sits on. Your agent can now call functions, query databases, fetch APIs, run code, or retrieve documents. Without tools, an agent is limited to reasoning over what's already in its context window.

Example:

- "What's the weather?" → Call weather API
- "Find user data" → Query database
- "Run this test" → Execute code

When to Use It

- ☒ The task requires current information or external state
- ☒ The agent needs to interact with systems outside its context
- ☒ You need real-time data or dynamic decision-making
- ☒ Almost every real-world task (tools are nearly mandatory)

The Trade-Off

Tool design matters. Poorly designed tools with unclear contracts confuse the agent. Clear inputs, outputs, and descriptions are critical.

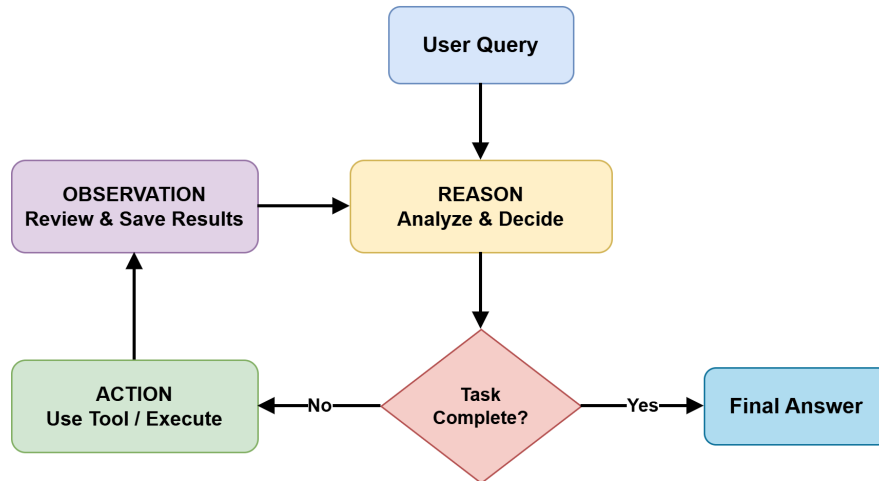
Best Practice

Tools don't change the reasoning pattern. A ReAct agent with tools is still ReAct. A planning agent with tools is still planning. Tools are the capability layer underneath your reasoning layer.

3. ReAct Pattern

For tasks where you only know the starting point

ReAct Pattern



What It Does

Reason → Act → Observe → Repeat. The agent thinks about what to do, takes action using tools, sees the result, and decides what comes next. It figures out the path step-by-step, adapting based on feedback.

Example: Customer support

1. "I received the wrong item"
2. Agent reasons: "I need to check the order"
3. Agent acts: Query order database
4. Agent observes: Found order #12345, wrong SKU
5. Agent reasons: "I should check warranty and offer replacement"
6. Agent acts: Offer replacement options

When to Use It

- ✓ The solution path isn't known in advance
- ✓ The task is exploratory (research, debugging, troubleshooting)
- ✓ Each step depends heavily on the previous result
- ✓ You need flexibility to handle unexpected situations
- ✓ Speed is acceptable (multiple iterations are fine)

The Trade-Off

ReAct can loop indefinitely if the stopping condition isn't clear. Agents can revisit the same questions, waste compute, and spiral. You need good task definition and clear stopping criteria.

Common Problem

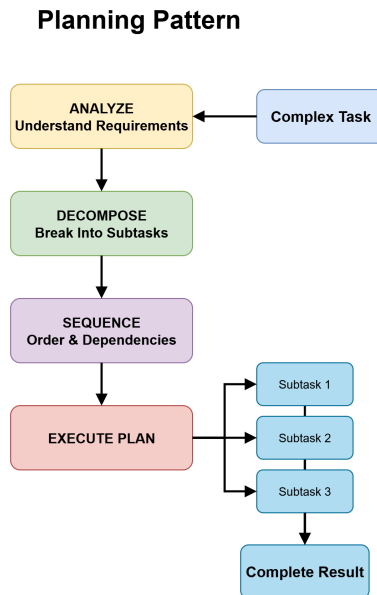
Excessive looping — agent keeps trying the same things. Fix: Add clearer stopping conditions, structure tools better, or consider switching to planning.

Best Practice

Define what "done" looks like. Give the agent a clear success condition. Structure tool outputs so they provide real information that moves the agent forward.

4. Planning Pattern

For complex tasks with identifiable structure



What It Does

Before executing, the agent creates a plan: "I'll do A, then B, then C." The plan defines stages and dependencies. Then the agent executes each stage, handling local details with ReAct-style reasoning as needed.

Example: Building a feature

1. Plan: Design → Implement → Test → Deploy
2. Execute: Agent designs the API, implements code, writes tests, pushes to production
3. Each stage can adapt, but the overall structure is set

When to Use It

- ☒ Task structure can be identified upfront
- ☒ Major dependencies are clear
- ☒ You want to avoid mid-execution surprises
- ☒ The task is large or multi-stage

The Trade-Off

Planning adds upfront cost and rigidity. If the real-world conditions differ from the plan, the agent gets stuck. If structure only emerges during execution, planning becomes misleading.

Common Problem

Plan abandonment — agent creates a plan then ignores it. This means the task is less structured than assumed. Switch to ReAct-only or use lightweight planning.

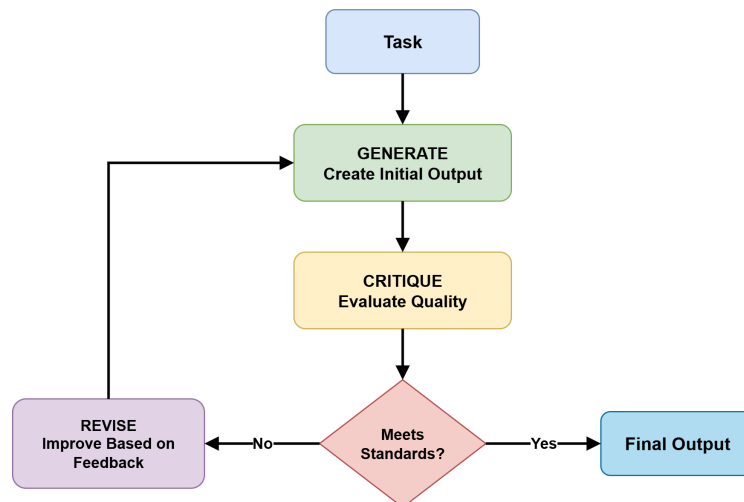
Best Practice

Use planning for tasks with clear structure. Use ReAct within each stage. This hybrid approach gets the best of both: upfront clarity plus execution flexibility.

5. Reflection Pattern

For high-stakes outputs where quality trumps speed

Reflection Pattern



What It Does

Generate an output, then critique it. The agent evaluates: "Is this correct? Does it meet the criteria? What's wrong?" If issues exist, the agent refines and tries again. This **generate** → **critique** → **refine** cycle improves output quality.

Example: Code generation

1. Agent writes a function
2. Critic checks: "Does this handle edge cases? Is it efficient? Does it follow style guidelines?"
3. If issues found, agent refines
4. Repeat until quality threshold met

When to Use It

- ☒ Output quality matters more than speed
- ☒ Clear evaluation criteria exist (valid SQL, correct logic, completeness)
- ☒ Errors are expensive (deployed code, client-facing documents)
- ☒ Latency is acceptable

The Trade-Off

Reflection adds latency and cost (extra model calls). It doesn't help when:

- Evaluation criteria are vague ("make it sound better")
- Speed is critical (live chat, real-time systems)
- The critic is too similar to the generator (it just agrees)

Common Problem

Reflection not improving output — critique cycles don't fix anything. Usually because evaluation criteria are unclear or the critic is too aligned with the generator. Use a different model for critique or be more explicit about what "good" means.

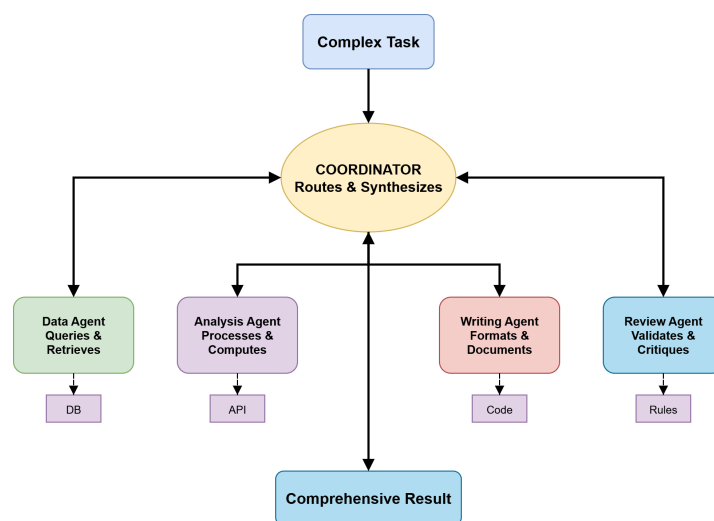
Best Practice

Make critique independent. Use a different system prompt or even a different model for evaluation. Be specific about quality criteria: "Check for SQL syntax, performance, and security vulnerabilities."

6. Multi-Agent Pattern

For tasks too big or specialized for one agent

Multi-Agent Pattern Example



What It Does

Decompose the problem into specialized agents. A coordinator routes tasks to specialists, agents work in parallel or sequence, and results combine back together.

Example: Document analysis

- **Coordinator:** Decides what needs doing
- **Legal agent:** Extracts and interprets legal clauses
- **Financial agent:** Analyzes pricing and terms
- **Risk agent:** Flags red flags
- Results merge into one comprehensive report

When to Use It

- ☒ Different parts need different expertise (legal vs. financial vs. technical)
- ☒ Work can happen in parallel to save time
- ☒ A single agent's context window is too small
- ☒ Specialization improves quality

The Trade-Off

Multi-agent adds complexity: coordination overhead, shared state management, more failure points, and routing decisions. Most single-agent systems fail not because they need multiple agents, but because they were over-engineered.

Common Problem

Routing failures — coordinator sends tasks to the wrong agent, or agent outputs don't combine well.
Fix: Use deterministic routing rules for predictable cases instead of LLM routing.

Best Practice

Only use multi-agent when you have a clear bottleneck that specialization actually solves. Ask: "Does this task genuinely need different reasoning styles, or am I just adding complexity?"